

WaveCraft

Interactive DSP Audio Workbench

2305028 – Atish Dipankar Paul
2305029 – Md Wasif Haque Fahim

Department of Computer Science and Engineering

CSE 220

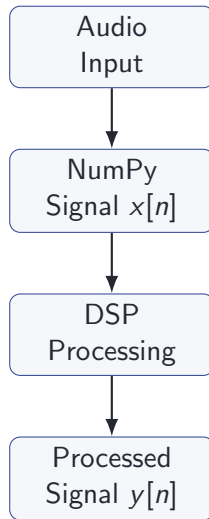
Project Overview

What is WaveCraft?

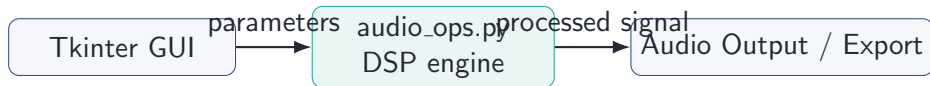
An interactive desktop audio-processing application where digital audio is represented as a discrete-time signal $x[n]$ and transformed using DSP algorithms.

$$y[n] = T\{x[n]\}$$

- Load / record audio
- Process the signal
- Visualize waveform and spectrum
- Listen to the result
- Export the processed audio



System Architecture and Feature Map



- Trim / reverse / fade
- Convolution and echo
- Space simulation
- FFT analysis
- Parametric EQ

- Spectral noise subtraction
- Frequency decomposition
- Speaker identification
- Doppler analysis
- BPM detection
- Audio stress analysis
- Pulse-echo ranging

Feature: Convolution and Echo I

An audio effect can be represented by an impulse response $h[n]$.

$$y[n] = x[n] * h[n]$$

For an echo:

$$h[n] = \delta[n] + g\delta[n - d]$$

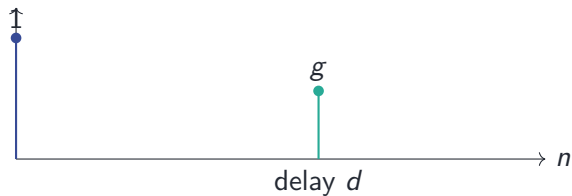
Therefore,

$$y[n] = x[n] + gx[n - d]$$

where

- d = delay in samples
- g = echo gain

Feature: Convolution and Echo II



Echo: Main Implementation I

The important implementation in `audio_ops.py` is:

```
delay_samples = int(delay_sec * sr)

h_len = delay_samples * n_repeats + 1
h = np.zeros(h_len)

h[0] = 1.0

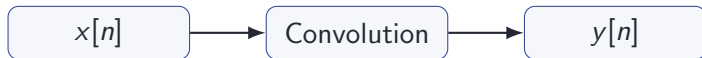
for k in range(1, n_repeats + 1):
    h[k * delay_samples] = decay ** k

y = fftconvolve(data, h, mode="full")
```

Conceptually:

$$h[n] = \delta[n] + g\delta[n - d] + g^2\delta[n - 2d] + \dots$$

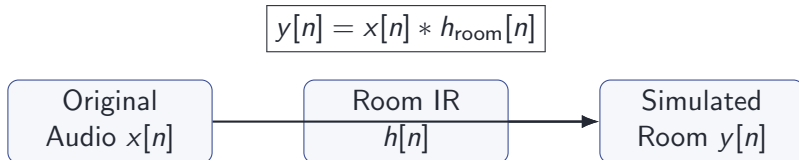
Echo: Main Implementation II



Each impulse in $h[n]$ produces a delayed, attenuated copy.

Feature: Space Simulator I

Instead of creating a synthetic echo, the application can use a recorded room impulse response.



The code resamples the impulse response when its sample rate differs from the track:

```
if ir_sr != sr:
    n_target = int(
        round(len(ir) * sr / ir_sr)
    )
    ir = scipy.signal.resample(
        ir, max(n_target, 1)
    )
```

Feature: Space Simulator II

Then:

```
wet_signal = fftconvolve(  
    data, ir, mode="full"  
)
```

Feature: FFT and Frequency Analysis I

The FFT changes the representation from

$$x[n]$$

to frequency components

$$X[k].$$

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

The application computes:

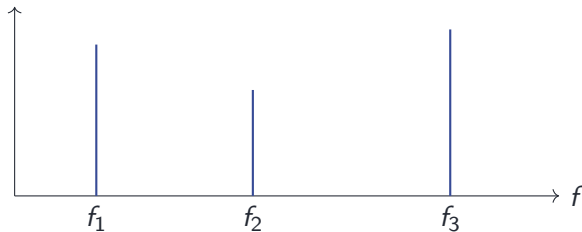
Feature: FFT and Frequency Analysis II

```
freqs = np.fft.rfftfreq(  
    n, d=1 / sr  
)  
  
spectrum = np.fft.rfft(data)  
  
magnitude = np.abs(spectrum) / n
```

The positive-frequency spectrum runs from

$$0 \rightarrow \frac{f_s}{2}.$$

Feature: FFT and Frequency Analysis III



Peaks indicate strong frequency components.

Feature: Parametric Multi-Band EQ I

The equalizer operates in the frequency domain.
For each frequency bin:

$$Y[k] = X[k]H[k]$$

The gain in dB is converted to amplitude:

$$A = 10^{G_{\text{dB}}/20}$$

Example:

$$+6 \text{ dB} \Rightarrow 10^{6/20} \approx 2$$

Feature: Parametric Multi-Band EQ II



The important code is:

```
Y = X * gain_curve  
result = np.fft.irfft(Y)
```

Feature: Spectral Subtraction I

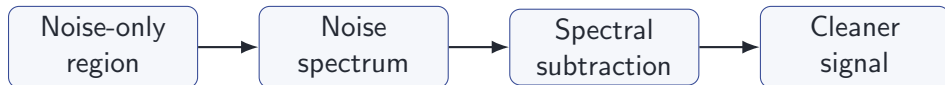
The application first learns the spectrum of a noise-only region.

$$N[k] = \text{average noise magnitude}$$

For each frame:

$$M_{\text{clean}}[k] = \max(M[k] - \alpha N[k], \text{floor} \cdot M[k])$$

where α controls subtraction strength.



Main implementation:

Feature: Spectral Subtraction II

```
clean_mag = mag - \  
    strength * noise_profile[:, None]  
  
clean_mag = np.maximum(  
    clean_mag,  
    floor * mag  
)
```

Feature: Frequency-Domain Filtering I

The application implements low-pass, high-pass, band-pass and band-stop operations by modifying FFT bins.

General form:

$$Y[k] = X[k]H[k]$$

For low-pass:

$$H_{LP}(f) = \begin{cases} 1, & f \leq f_c \\ 0, & f > f_c \end{cases}$$

For high-pass:

$$H_{HP}(f) = \begin{cases} 0, & f < f_c \\ 1, & f \geq f_c \end{cases}$$

Feature: Frequency-Domain Filtering II

Main implementation:

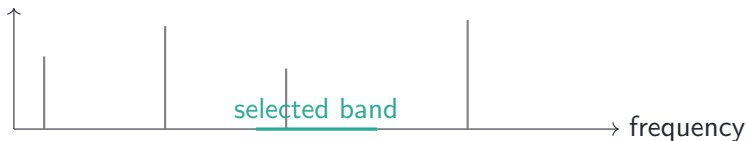
```
freqs = np.fft.rfftfreq(  
    n, d=1 / sr  
)  
  
spectrum = np.fft.rfft(data)  
  
spectrum[freqs > cutoff_hz] = 0  
  
result = np.fft.irfft(  
    spectrum, n=n  
)
```

Feature: Frequency Decomposition

A signal can be isolated approximately by selecting a frequency region.

$$X_{\text{selected}}[k] = X[k]H[k]$$

where $H[k]$ is a binary frequency mask.



The project also contains a Demucs-based stem-separation path, which is different from simple FFT masking: it uses a neural network to separate musical stems.

Feature: Voice and Speaker Identification

The speaker-identification pipeline extracts an MFCC voiceprint.

$x[n] \rightarrow \text{spectral analysis} \rightarrow \text{Mel filterbank} \rightarrow \text{MFCC vector}$

The application stores an MFCC vector for each enrolled speaker.



Relevant functions:

```
extract_mfcc(...)  
identify_speaker(...)
```

The identification compares the current MFCC vector with stored voiceprints.

Feature: Doppler Speed Shift I

A moving sound source produces a frequency shift.
The application estimates:

f_a = approaching frequency

f_d = departing frequency

and uses

$$v = c \frac{f_a - f_d}{f_a + f_d}$$

where c is the speed of sound.

Feature: Doppler Speed Shift II



The project automatically finds the loudest point of the recording and treats it as the closest pass-by point.

Feature: BPM / Pulse Detection I

The application estimates periodicity from the amplitude envelope.

Pipeline:

$x[n] \rightarrow \text{rectification} \rightarrow \text{smoothing} \rightarrow \text{autocorrelation} \rightarrow \text{period} \rightarrow \text{BPM}$

If the detected period is T seconds:

$$BPM = \frac{60}{T}$$

For a sample lag L at sample rate f_s :

$$T = \frac{L}{f_s}$$

therefore

Feature: BPM / Pulse Detection II

$$BPM = \frac{60f_s}{L}$$

Main function:

```
compute_bpm(data, sr, ...)
```

The code creates an onset/energy envelope and searches for the most plausible periodic lag.

Feature: Audio Stress Analysis I

The application measures two forms of frame-to-frame voice instability:

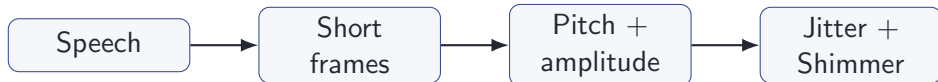
Jitter $\rightarrow$ pitch-period variation

Shimmer $\rightarrow$ amplitude variation

For periods T_i :

$$Jitter \propto \frac{\text{variation}(T_i)}{\text{mean}(T_i)}$$

The implementation first estimates pitch using autocorrelation on short frames.



Feature: Audio Stress Analysis II

Main function:

```
analyze_audio_stress(  
    data, sr,  
    frame_ms=30.0,  
    hop_ms=15.0  
)
```

Feature: Pulse-Echo Ranging I

This feature is the audio analogue of sonar/radar.
A known pulse/chirp is emitted:

$$s(t)$$

The received signal contains a delayed copy:

$$r(t) \approx \alpha s(t - \tau)$$

Cross-correlation finds the delay:

$$R_{sr}(\tau) = \sum_n s[n]r[n + \tau]$$

Once the delay is known:

Feature: Pulse-Echo Ranging II

$$d = \frac{c\tau}{2}$$

The factor 2 occurs because the sound travels to the object and back.

Pulse-Echo: Main Implementation I

The application generates a chirp:

```
generate_chirp(  
    sr=44100,  
    duration_sec=0.02,  
    f0=2000.0,  
    f1=9000.0  
)
```

Then correlates the emitted and received signals.

```
corr = scipy_correlate(  
    received,  
    emitted,  
    mode="full"  
)
```

Pulse-Echo: Main Implementation II

The important conceptual pipeline is:

chirp $\rightarrow$ speaker $\rightarrow$ reflection $\rightarrow$ microphone $\rightarrow$ correlation $\rightarrow \tau \rightarrow d$

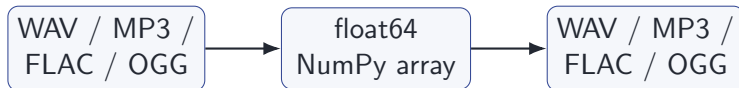


Feature: Audio Format Conversion

The application internally works with a common representation:

mono float64 array in $[-1, 1]$

This makes DSP operations independent of file format.



For WAV:

```
int_data =  
    (data * 32767).astype(np.int16)  
  
wavfile.write(  
    path, sr, int_data  
)
```

Other formats use Pydub + FFmpeg.

Additional DSP Features

Time Domain

- Trim
- Reverse
- Scale / volume
- Fade in/out
- Moving-average smoothing
- Echo

Frequency Domain

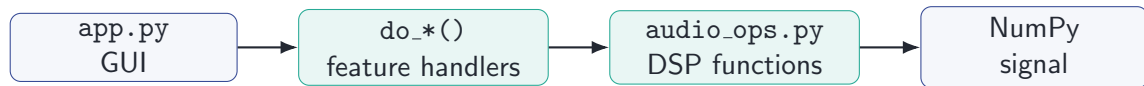
- FFT spectrum
- Low-pass
- High-pass
- Band-pass
- Band-stop
- Parametric EQ
- Spectral subtraction

A common frequency-domain structure is:

$$x[n] \xrightarrow{\text{FFT}} X[k] \xrightarrow{\times H[k]} Y[k] \xrightarrow{\text{IFFT}} y[n]$$

This same idea appears repeatedly throughout the project.

Important Code Architecture



For example, echo processing uses:

`do_echo()`

↓

`ao.add_echo()`

↓

`fftconvolve(data, h)`

This separation makes the DSP algorithms independently testable.

Overall DSP Pipeline



Audio $\rightarrow x[n] \rightarrow T\{x[n]\} \rightarrow y[n] \rightarrow$ Audio

The project therefore demonstrates both:

- classical DSP algorithms, and
- signal-analysis / ML-assisted features.

Key DSP Concepts Demonstrated

$$y[n] = x[n] * h[n]$$

Convolution

$$Y[k] = X[k]H[k]$$

Frequency-domain processing

$$X[k] = \sum_n x[n]e^{-j2\pi kn/N}$$

FFT

$$A = 10^{G/20}$$

dB gain

$$BPM = \frac{60f_s}{L}$$

Periodic pulse detection

$$d = \frac{c\tau}{2}$$

Pulse-echo ranging

$$y[n] = x[n]e[n]$$

Fade envelope

What the project demonstrates

WaveCraft combines a graphical audio editor with practical digital signal-processing techniques.

- Audio represented as discrete-time samples
- Time-domain signal processing
- FFT-based frequency-domain processing
- Convolution and impulse-response modelling
- Noise reduction and equalization
- Pitch, BPM and Doppler analysis
- Voice and speaker analysis
- Pulse-echo ranging

Signal → DSP → Analysis → Audible Result